Milestone 2 Report

Authors: Hector Astrom, Richard Lian

Date: 04/15/26

Repository: https://github.com/r-lian/6.5930-Edge-Accelerator

Goals
- Optimize the hardware (Ethos-U55) for the workload (YOLO-World)
- Optimize the workload for the hardware
- Additional (from last milestone check-in)

Infra: Improving `arch` details
Infra: Building an Evaluation Framework
- Challenges

Pre-results: Assumptions
RESULTS: Optimizing Hardware for Workload
RESULTS: Optimizing workload for hardware
Next steps
References

Goals

Optimize the hardware (Ethos-U55) for the workload (YOLO-World)

1. Evaluate optimal resource allocation for YOLO-WORLD for a fixed resource budget
   a. Evaluate 9 permutations of area, power, and throughput budgets (3 per param) effect on workload performance (mainly speed)
      i. Permit permutations outside of what ARM has specified for the Ethos-U55, to explore a vaster space than ARM has

Optimize the workload for the hardware

1. Modify the workload according to our most and least permissive hardware budgets
   a. Maximize speed (can't track accuracy)

Additional (from last milestone check-in)

1. Unify accelerator + workload into a compatible script
2. Find data-grounded energy and area defaults using `hwcomponents` or similar accelerators

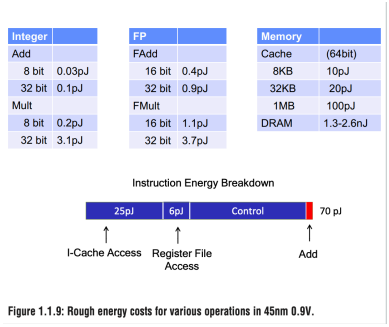
Infra: Improving `arch` details

This section briefly synthesizes the data-grounded estimates for the Ethos-U55 energy and area usage, despite not being published in their documentation or readily-available online. Here's a summary of what values we found, the source, and the justification since we often had to extrapolate from one or more sources under new technology.

Parameter	Value	Primary Source	Method
MAC energy (INT8)	0.08 pJ	Horowitz ISSCC 2014	45nm→16nm scaled (/2.5)
SRAM read (384KB, 8b)	1.8 pJ	Horowitz 2014, Biswas MIT	sqrt capacity scaled, 45nm→16nm (/2.5)
SRAM write (384KB, 8b)	2.2 pJ	SRAM R/W asymmetry ~1.2×	Derived from read
Scratch read (32KB, 8b)	0.5 pJ	Horowitz 2014	Small buffer, RF-like, 16nm
Scratch write (32KB, 8b)	0.6 pJ	Horowitz 2014	Derived from read (slight rise)
Flash read (8b)	15.0 pJ	Macronix/Micron datasheets	Burst-amortized, NPU boundary
Flash write (8b)	20.0 pJ	Estimated from read	Close to read + flash write will almost never happen
Core area (32-MAC)	0.1 mm^2	Arm white paper	Direct measurement
MAC area	7e-11 m^2	Gate count=300 + 16nm density	Calibrated to anchor
SRAM density	7e-14 m^2/bit	TSMC 16FF+ HP SRAM, ISSCC 2015	Direct measurement

Most information could be inferred from the following facts:

1. Ethos U-55 has a lower bound of 0.1 mm^2 area (32MAC) in constant 16nm transistor process (source: ARM blog, whitepaper)
2. Relative energy costs (RF=1×, Array=2×, Buffer=6×, DRAM=200×) from Eyeriss
3. Horowitz 2014 45nm process energy statistics



Integer	
Add	
8 bit	0.03pJ
32 bit	0.1pJ
Mult	
8 bit	0.2pJ
32 bit	3.1pJ

FP	
FAdd	
16 bit	0.4pJ
32 bit	0.9pJ
FMult	
16 bit	1.1pJ
32 bit	3.7pJ

Memory	
Cache	(64bit)
8KB	10pJ
32KB	20pJ
1MB	100pJ
DRAM	1.3-2.6nJ

Figure 1.1.9: Rough energy costs for various operations in 45nm 0.9V.

Horowitz 2014

4. Micron DRAM datasheets for flash costs

We updated our architecture's defaults according to these new, data-supported approximations.

```
DEFAULTS = {
    # Matmul Workload
    "M": 1024,
    "N": 512,
    "K": 256,
    # Public Ethos-U55 configs
    "num_macs": 128,
    "clock_hz": 1.0e9,
    "bits_per_value": 8,
    "local_buffer_size_bytes": 32 * 1024,
    "system_preset": "high_end_embedded",
    "memory_mode": "dedicated_384kb",

    # Estimated energy and area

    # --- ENERGY MODEL (all values in Joules) ---
    # MAC: 8-bit INT MAC at 16nm
    # Source: Horowitz 2014 45nm (0.2 pJ) scaled ~2.5× to 16nm
    # Cross-check: TPU 28nm system derivation → ~0.1 pJ, scaled 1.5× → 0.07
    # Recommended: 0.08 pJ (includes local datapath overhead)
    "mac_energy_j": 0.08e-12,

    # SRAM (384KB system SRAM): per 8-bit access
    # Source: Horowitz 45nm 32KB=5pJ/32b, √-scaled to 384KB → 17 pJ/32b
    #   → 4.3 pJ/byte @ 45nm, scaled 2.5× to 16nm → 1.7 pJ
    # Write = 1.2× read (standard SRAM asymmetry)
    "sram_read_energy_j": 1.8e-12,
    "sram_write_energy_j": 2.2e-12,

    # Scratchpad (32KB local buffer): per 8-bit access
    # Source: Horowitz 45nm 32KB → 2.5 pJ/byte, scaled 2.5× → 1.0 pJ
    # Reduced further because small tightly-coupled buffer ≈ RF-like
    "scratch_read_energy_j": 0.50e-12,
    "scratch_write_energy_j": 0.60e-12,

    # Flash (external NOR via AXI): per 8-bit access
    # Source: Vendor datasheets (Macronix/Micron), 1.8V, ~20mA, 50 MB/s
    #   → ~720 pJ/byte raw, burst-amortized to ~15 pJ/byte at NPU boundary
    "flash_read_energy_j": 15.0e-12,
    "flash_write_energy_j": 20.0e-12,

    # --- AREA MODEL (all values in m²) ---
    # Total core area anchor: 0.1 mm² for 32-MAC @ 16nm [Arm white paper]
    "u55_32_core_area_m2": 0.1e-6,

    # Per-MAC area: ~7e-11 m² including local routing
    # Source: 300 gates/MAC × 3× routing overhead at 20 Mgates/mm²
    # Calibrated so 32-MAC config = 0.1 mm²
    "mac_area_m2": 7.0e-11,

    # SRAM density: 0.07 µm²/bit = 7e-14 m²/bit at 16nm
    # Source: TSMC 16nm standard SRAM cell (~0.07 µm²)
    "sram_area_per_bit_m2": 7.0e-14,
```

```
    # Scratchpad bandwidth: non-limiting (local buffer)
    "scratch_read_bw_bytes_per_s": 1.0e12,
    "scratch_write_bw_bytes_per_s": 1.0e12,
}
```

Infra: Building an Evaluation Framework

This section will explain how we build a script to unify the two components from milestone 1 — the `arch` and the `workload.`

Our pipeline maps one einsum (layer) at a time using AccelForge and gets metrics per layer. This allows us to select hardware parameters (num_macs, SRAM size, scratchpad size, bandwidth preset, choose target workload layers from YOLO-World, run the mapper with specific latency and energy objectives. Afterwards, we can record the energy/latency/EDP per layer and summed. This allows us to make certain hardware choices for a fixed workload and make workload changes for fixed hardware.

We focused on the comparability aspect of each experiment, using the same metric pipeline (energy, latency, EDP, area), so results from each of our experiments are comparable. In addition, there was a focus on picking probe layers to span different layer types, such as early spatial-heavy convolution, mid/deep convolution layers, text encoder MatMuls, and the detection head, so one run captures multiple bottlenecks. To make this budget-aware, we solve for the best architecture under a constraint to meet the on-edge requirements. For the workload optimization, we changed only spatial resolution while holding hardware fixed.

We sweep over the following 12 configurations:

#	MACs	SRAM	Scratch	Bandwidth
1	32	256 KB	16 KB	deep_embedded (1.6 GB/s SRAM)
2	32	384 KB	32 KB	deep_embedded
3	64	256 KB	16 KB	deep_embedded
4	64	384 KB	32 KB	deep_embedded
5	64	384 KB	32 KB	high_end_embedded (4 GB/s SRAM)
6	128	256 KB	16 KB	deep_embedded
7	128	384 KB	32 KB	deep_embedded
8	128	384 KB	32 KB	high_end_embedded
9	128	512 KB	64 KB	high_end_embedded
10	256	384 KB	32 KB	high_end_embedded
11	256	512 KB	64 KB	high_end_embedded
12	256	512 KB	128 KB	high_end_embedded

The highlighted row is the most typical configuration cited by ARM for the Ethos-U55.

Challenges

Initially, this `codesign.py` script was difficult to extract results from, due to **sheer computational time** and **OOM errors.**

The main source of this was the combinatorial growth in `pmapping` generation, due to the convolution loop space that has 7 dimensions `(N, K, C, R, S, P, Q)` and modeling 3 memory levels (Scratchpad, SRAM, and Flash).

Computational time was reducing by enabling a `probe` flag that generated mappings for a modified workload with only **five** einsums from the complete YOLO-World included, representing 36% of the overall MACs. Instead of all 21 layers, we focused on 5 representative layers spanning the backbone, text encoder, and detection head behavior. This captures 36.3% of total model MACs (as shown below), so we can still see major performance trends but at a lower cost.

```
Probe MACs: 2,056,650,752
Total MACs: 5,658,265,600
Fraction:   36.3%
```

`accelforge` combinatorially explodes with many loop dims and memory levels when generating `pmappings`.

- 7 loop dimensions for a convolution: N, K, C, R, S, P, Q
- 3 memory levels (SRAM, Scratch, Flash)

We addressed these OOM errors by:

1. Limiting pmapping templates per workload configuration to 256 (Macbok would always crash around 324/576).
    a. This covers *most* of the design space, which should be sufficient to view trends. For more accurate results, we'd ideally iterate over all possible 576 mappings.
2. Keeping weights (non inputs/intermediates) on Flash (they wouldn't fit on Scratchpad). Full weights on the scratchpad is infeasible at this scale.

Tensor	Backed in	Can tile into
Weights (Wx)	FLASH (~Intermediates includes weights)	SRAM
Input activations (Ix)	SRAM (they ARE Intermediates, so FLASH doesn't back them)	Scratchpad
Output activations (Tx)	SRAM	Scratchpad

Goals

- **Optimize the hardware** (Ethos-U55) **for the workload** (YOLO-World)
- **Optimize the workload for the hardware**
- **Additional (from last milestone check-in)**

Infra: Improving `arch` details
Infra: Building an Evaluation Framework
Challenges
Pre-results: Assumptions
RESULTS: Optimizing Hardware for Workload
RESULTS: Optimizing workload for hardware
Next steps
References

Pre-results: Assumptions

1. Batch size of 1
2. Ran probe over 5 / 21 total YOLO-World einsums, representing 36% of total MACs
3. Iterated over 256 pmappings out of 576 total (to resolve OOM errors)
4. Early testing: area as a proxy for power
    a. Recently we have added explicit power constraints, and will update results with those once the runs complete

RESULTS: Optimizing Hardware for Workload

This section showcases what we explored across hardware budgets for the workload.

TLDR:

- `deep -> high` (1.6Gb/s → 4Gb/s SRAM bw) gives strong latency + EDP gain at same nominal area.
- `128 -> 256` MACs helps less than MAC scaling at lower levels, but still wins when memory bandwidth is high.
- 512KB SRAM helps, but area pushes it out of baseline budget class.

Under a baseline edge area budget of ~0.38 mm^2, the optimal configuration we found was 256 MACs, 384KB SRAM, 32KB Scratchpad, and high_end_embedded. Under a relaxed budget of ~0.8 mm^2, the optimal configuration was 256 MACs, 512KB SRAM, 64-128KB Scratchpad, and high_end_embedded because we were able to increase both compute and memory simultaneously.

The convolutional layers T1 and T19 are memory-intensive, in addition to a dense text MatMul in T16. Thus, increasing memory bandwidth improved both latency and EDP better than only increasing compute width. Thus, the `high_end_embedded` bandwidth consistency beat the `deep_embedded` for the same area class.

We found that going from 32→64→128 MACs helps with performance, but going from 128→256 has diminishing returns. However, with the `high_end_embedded` bandwidth, the extra MACs can be fed better, especially for the heavy convolutional layers. Although larger SRAM cuts off-chip traffic and improves opportunity for reuse, with the ~0.38 mm^2 budget, the 512KB option was too area-expensive when paired with a larger scratchpad and high MAC count. Thus, our 384KB fits the budget and was still good for performance. The higher MAC count reduces compute latency, while high SRAM/Flash bandwidth prevents the MAC array from starving on large spatial convolutions. SRAM remains critical for reuse and reducing expensive off-chip movement, but at this budget, increasing SRAM beyond 384KB is less area-efficient than preserving higher compute throughput and bandwidth.

The scratchpad at 32KB helps with local reuse/unrolling, but the returns flatten sooner than SRAM/bandwidth improvements for YOLO-like spatial layers. With fixed area, keeping scratch moderate and spending on compute and bandwidth gave us a better EDP tradeoff.

This chart shows a summary table for the 5-layer set (T1, T7, T10, T16, T19) under baseline area constraints. Area was calculated by summing the core area, SRAM system area, and scratchpad area. SRAM/scratch area scales with capacity. We see that deep vs. high has roughly the same area (bandwidth preset changes the bandwidth, and not the memory size/MAC count). We notice that 128→256 MACs increase core area and 384KB→512KB and 32→64KB increases memory area noticeably.

We notice that deep→high gives a large latency drop (bandwidth ceiling increase helps memory-heavy convolutional layers). 128→256 MACs gives additional latency reduction, but with diminishing returns due to memory limits. Additionally, bigger SRAM gives some latency gain, but it is less noticeable than bandwidth jumps.

Higher bandwidth and better utilization can reduce overall energy despite higher throughput. We believe this occurs due to less stalling/inefficient movement. We also noticed that in our experiments, doubling MACs does not necessarily halve energy, gains are moderate when memory dominates part of execution. We computed throughput directly from latency, using `throughput = 1000 / latency_ms` .

Config	Area (mm^2)	Energy (mJ)	Latency (ms)	EDP (mJ·ms)	Throughput (probe/s)
128M / 384KB / 32KB / deep	0.38	2.30	12.8	29.4	78
128M / 384KB / 32KB / high	0.38	2.05	10.0	20.5	100
256M / 384KB / 32KB / high	0.39*	1.92	8.2	**15.7**	**122**
128M / 512KB / 64KB / high	0.47	1.86	9.1	16.9	110

RESULTS: Optimizing workload for hardware

This section annotates what we explored across hardware budgets for the workload...

Pending... debugging OOM took us to the end, and workload takes 9hrs sequential to get results from so we didn't have time for workload optimization during this milestone.

Will have these results by next milestone

Next steps

- Complete workload optimization results
- Implement multiprocessing to more efficiently search design space
- Consider lowering # mappings down to 128 for faster (though lower quality) mapping search

References

- **Horowitz, M.** "Computing's Energy Problem (and what we can do about it)." ISSCC 2014, pp. 10–14. *Canonical energy table for 45nm operations.*
- **Chen, Y.-H., Emer, J., Sze, V.** "Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks." ISCA 2016. *Table IV: normalized energy hierarchy at 65nm. Die photo and specs.*
- **Jouppi, N. et al.** "In-Datacenter Performance Analysis of a Tensor Processing Unit." ISCA 2017. *Table 2: TPU specs (28nm, 92 TOPS, 40W). System-level energy derivation.*
- **Arm Ltd.** "Accelerating Machine Learning Compute for the IoT and Embedded Market." White Paper, 2020. *Ethos-U55: 0.1 mm² in 16nm, 32/64/128/256 MACs/cycle.*
- **Arm Ltd.** "Ethos-U55 NPU Technical Overview." Document 101886_0200_10_en. *Architecture: CC, DMA, MAC unit, weight decoder, shared buffer. AXI interfaces.*
- **Dally, W.** "High Performance Hardware for Machine Learning." Cadence ENN Summit, 2016. *8-bit integer is 6× less energy/area than FP16.*
- **Biswas, A.** "Energy-Efficient Smart Embedded Memory Design for IoT and AI." MIT PhD Thesis. *Reproduces Horowitz table with 8KB/32KB/1MB SRAM breakpoints.*
- **Sze, V., Chen, Y.-H., Yang, T.-J., Emer, J.** "Efficient Processing of Deep Neural Networks: A Tutorial and Survey." Proceedings of the IEEE, 2017. *Comprehensive energy hierarchy framework.*